

מערכת לזיהוי כשרות ופגמי אתרוג – סקירה מעשית

תקציר: בניית מערכת לזיהוי אוטומטי של כשרות אתרוגים מחייבת שילוב עבודה קפדנית באיסוף נתונים (תמונות ממספר זוויות), יצירת מודלים תלת־ממדיים בפוטוגרמטריה, תיוג תכונות הלכתיות, ופיתוח מודלי למידת מכונה (Classification / Supervised Learning) לזיהוי פגמים והערכת כשרות. להלן נפרט תהליך מלא – משלב איסוף התמונות וארגון המאגר, דרך יצירת מודל 3D איכותי ותיוג הלכתי מדויק, עד למבנה מערכת **הזיהוי האופטימלי** ולכלים המומלצים. בנוסף, נציג דגשים ארגוניים לעבודת צוות רוחבי (Cross-Functional) וניהול גרסאות הנתונים.

מאגר הנתונים: איסוף, צילום וארגון התמונות

צילום מקיף ומגוון: כדי לבנות מאגר נתונים אמין, יש לצלם כל אתרוג ממספר זוויות באופן שיכסה את כל היקף הפרי. מומלץ לצלם ב-360° סביב האתרוג במספר גבהים (למשל זוויות צדדיות וגם מלמעלה/מלמטה), כך שכל נקודה על פני האתרוג תופיע בלפחות 3 תמונות עם חפיפה של כ-66% בין תמונה לתמונה ¹. שיטה זו מבטיחה כיסוי מלא של שטח הפרי ומאפשרת גם בניית מודל תלת־ממדי איכותי. בפועל, ניתן להניח את האתרוג בתא צילום (Lightbox) עם רקע חלק, ולהשתמש בצילום רציף על גבי **פליטה מסתובבת**: מסובבים את האתרוג כמה מעלות בין צילום לצילום, עד להשלמת 360°. כך נצברות 50~ תמונות לכל סיבוב מלא ². יש לחזור על התהליך במספר תנוחות של האתרוג (על צידו, על הראש וכו') כדי לחשוף גם את האזור שעליו הוא נח ³. באופן זה ייתכן שה"כ 150~200 תמונות לאתרוג (3~4 סטים של 50 תמונות בזוויות שונות).

ציוד ותנאי צילום: רצוי להשתמש במצלמה איכותית (DSLR/אלא-מראה, או סמארטפון מתקדם) המספקת תמונות ברזולוציה גבוהה, על חצובה לייצוב. בתאורה – עדיף אור מפוזר (**diffused**) למניעת צללים חדים. תא צילום ייעודי עם רקעים מתחלפים (לבן/שחור) מסייע ליצור רקע אחיד שקל להפרדה. יש לוודא פוקוס חד על כל פני הפרי: מומלץ לצלם במצב **ידני** עם מפתח צמצם גבוה (f/10 ומעלה) כדי להגדיל עומק שדה – כך גם חלקים קרובים וגם רחוקים מהמצלמה יהיו בפוקוס ⁴. הגדרות אלה יבטיחו שהמרקם המוחספס והפגמים הקטנים יופיעו בצורה חדה בתמונות.

ארגון המאגר ותיוג ראשוני: צוות איסוף הנתונים (צלמים ו"קרוסרים" – ייתכן הכוונה לצוותי שטח רוחביים) צריך לקבוע שיטת ארגון עקבית לתמונות. מקובל ליצור תיקיית-משנה לכל אתרוג (למשל תיקייה בשם `Etrog_0123` המכילה את כל תמונותיו). שמות הקבצים בתוך התיקיה יכולים לכלול מזהה האתרוג והזווית או מספר רציף (למשל `Etrog_0123_01.jpg`, `Etrog_0123_02.jpg` וכו'). כך, **כל התמונות של פרי מסוים מקובצות ליחידה** וקל לשחזר מה שייך למה. בנוסף, רצוי לשמור יומן (spreadsheet) עם מטא-נתונים על כל אתרוג: מקור/זן, מצב כללי, הערות מהבודק ההלכתי וכו'. פורמט קובצי התמונות רצוי שיהיה ללא אובדן מידע – כגון **PNG** או **JPEG באיכות גבוהה** (אחוז דחיסה נמוך) – כדי שלא לפגוע בפרטים הקטנים.

כמות ודיוורסיטי (גיוון) התמונות: בכדי לאמן רשת עמוקה נדרשת כמות גדולה ומגוונת של דוגמאות. כעיקרון, **כמה שיותר – יותר טוב**; כלל אצבע הוא לפחות אלף תמונות לכל קטגוריה שנרצה לסווג ⁵. במקרה שלנו הקטגוריות עשויות להיות "כשר" מול "פסול", או סוגי פגמים (חזזית, חור, פיטם שבור וכו'). חשוב לגוון את התנאים: אתרוגים ממקורות שונים, בגדלים וגוונים שונים, וצילום תחת שינויים קלים בתאורה וברקע – כל זאת על מנת שהמודל ילמד לזהות פגמים **באופן כללי** ולא יותאם רק לתנאי צילום אחד. עם זאת, מאחר שהצילום לרוב מתבצע בסביבה מבוקרת (רקע חלק ותאורה קבועה), הגיוון העיקרי יבוא משונות הפריטים עצמם (צבעים, צורות ורמות פגם). במקרה של חוסר בדוגמאות, ניתן להשתמש ב-**Data Augmentation** (הרחבת נתונים מלאכותית) – טרנספורמציות על התמונות (סיבוב, שינוי תאורה, הגדלה/הקטנה וכו') להגדלת המגוון ⁶ – **אם כי בזהירות**: אין לעוות יותר מדי תצלום של אתרוג כי עלולים לטשטש או לשנות פגם קטן.

מבנה תיקיות ופורמט תיוג: לאחר צילום, מגיע שלב סידור התמונות והכנת קבצי התיוג המתאימים. מקובל במערכות **זיהוי עצמים** (Object Detection) לארגן את הנתונים כך שלכל תמונת קלט יהיה קובץ תיוג נפרד עם אותו שם קובץ (רק סימנת שונה). למשל, עבור `Etrog_0123_01.jpg` יהיה קובץ `Etrog_0123_01.txt` המכיל את האנוטציות

הפגמים באותה תמונה. בפורמט YOLO, למשל, כל שורת תיוג מורכבת ממספר קטגוריה ואחריו 4 מספרים לנורמליזציה תיבת התיחום (Bounding Box) של הפגם. **חשוב להקפיד על שמות קבצים סטנדרטיים ואחידים** ולשמור את קובצי התיוג במבנה תואם ⁷. ארגון ברור (למשל תיקיית `train` ו-`validation` עם תמונות וקובצי התיוג המתאימים) יבטיח תהליך אימון חלק של המודל.

דוגמה למבנה מאורגן:

```
dataset/
├── train/
│   ├── images/ (תמונות אימון)
│   └── labels/ (קבצי תיוג לאימון)
└── val/
    ├── images/ (תמונות אימות)
    └── labels/ (תיוגי אימות)
```

בכל תיקיית labels, כל קובץ txt נושא שם זהה לתמונה המתאימה ומכיל שורות תיוג (למשל: `0 0.45 0.60 0.10 0.05` - קטגוריה 0 עם תיבה במיקום וגודל מנורמל). פורמט עקבי זה חיוני להבנת הרשת מה לחפש ⁷.

יצירת מודלים תלת־ממדיים איכותיים מתמונות

אחד המרכיבים החדשניים בפרויקט הוא שימוש במודל **תלת־ממדי (3D)** של האתרוג, הנבנה מכל תמונותיו, לצורך ניתוח שלם של פני הפרי. מודל כזה מאפשר למדוד בצורה מדויקת מיקום של פגמים (למשל **האם פגם נמצא בשליש העליון של האתרוג**) ולהפיק המחשות ויזואליות בכיסוי 360° . להלן מתודולוגיית פוטוגרמטריה מומלצת:

תוכנות פוטוגרמטריה: ישנן מספר תוכנות נפוצות לבניית מודל 3D מתמונות חופפות. בקטגוריית הקוד הפתוח ניתן להשתמש ב-AliceVision Meshroom או ב-COLMAP. תוכנות אלה מקבלות את כל תמונות האתרוג (בדרכי פורמט JPEG/PNG) ומבצעות תהליך **Structure from Motion**: מזהות נקודות עניין חוזרות בתמונות, מחשבות את מיקום המצלמה בכל צילום, ובונות **ענן נקודות** תלת־ממדי. לאחר מכן מתבצע מיזוג ענן-נקודות צפוף, יצירת רשת משולשים (mesh) ובסוף הקרנת טקסטורה על המשטח. Meshroom, למשל, מספקת ממשק גרפי נוח שבו פשוט גוררים את התמונות לתוך הפרויקט - והתוכנה מחשבת את המודל אוטומטית ⁸. לחלופין, ישנן תוכנות מסחריות מובילות: Agisoft Metashape (Photoscan) ו-RealityCapture - הידועות בדיוק ובחדות התוצאות שלהן. RealityCapture, לדוגמה, יכולה לנצל GPU להאצת החישובים ומפיקה מודל + טקסטורה ברזולוציה גבוהה (ניתן לייצא ל-Obj, PLY וכו') ^{9, 10}.

תנאים קריטיים לאיכות מודל: איכות המודל תלויה מאוד באיכות וכמות התמונות. יש להקפיד על **חפיפה גבוהה** בין התמונות כפי שנזכר - כך שכל פרט על האתרוג נראה ממספר זוויות ¹. רקע חלק (לבן/שחור) מסייע כי תוכנות יכולות להתעלם ממנו (בחלקן קיים אפילו כלי Mask אוטומטי). מומלץ לצלם את האתרוג על רקע שאינו כולל פרטים מיותרים - כך האלגוריתם יתרכז בהתאמת נקודות על הפרי עצמו ¹¹. עוד כלל: **כיסוי 360° בכמה "רכבות"** - יש לצלם לפחות בשני גבהים: למשל מסלול תמונות סביב האמצע, ועוד מסלול בזווית אלכסונית מלמעלה, ואחד מלמטה. בנוסף, כדי לקבל גם את האזור הנסתר (נקודת המגע של האתרוג עם המשטח), יש לצלם אותו גם כשהוא הפוך או מוטה. שיטה אחת היא ליצור שני מודלים נפרדים (מצב רגיל והפוך) ולאחד אותם בתוכנה; שיטה שנייה, קלה יותר עם תוכנה תומכת, היא כפי שתואר לעיל: להשתמש ברקע חלק ולצלם במספר תנוחות, כשהתוכנה תופסת את הכל כסט אחד ^{12, 3}. לדוגמה, מצלמים אתרוג במצב עומד, אח"כ מניחים אותו על צידו ומצלמים סיבוב נוסף, ואולי גם הפוך (פיטם כלפי מטה) - ומזינים את כל התמונות יחד למנוע הפוטוגרמטריה. אם הסט נעשה נכון, ניתן להשיג כיסוי כמעט מלא של פני האתרוג.

פרמטרים ודיוק: כדי לקבל מודל **חד ומדויק** ישנה חשיבות לכמה פרמטרים: - **רזולוציית תמונה:** להשתמש במצלמה של 12MP ומעלה. בפרטים קטנים (כמו נקודה שחורה או חזית זעירה) - יותר פיקסלים = יותר סיכוי שייקלט. - **פוקוס ועומק שדה:** כפי שצוין, צמצם סגור (מס' f גבוה) למיקוד מלא של פני האתרוג ⁴. - **תאורה:** תמונות כהות או בהירות מדי לא

"יראו" פרטים בטקסטורה. יש לכוון את חשיפת המצלמה כך שפני האתרוג מוארים היטב ללא אזורים שרופים (overexposed) או חשוכים לגמרי. - **כיוול ועדשות:** אם משתמשים בעדשה רחבה, יש לעדכן את התוכנה בפרמטרים (רבות מהן יודעות לתקן עיוותי עדשה). תמונות נטולות עיוות יניבו מודל מדויק יותר בפרופורציות. - **מספר התמונות:** לא להתקמצן - עדיף עודף תמונות מאשר חסר. במקרה של אתרוג עם הרבה בליטות וחריצים, ייתכן צורך גם ב-200-300 תמונות כדי שהכל יופיע. הכלל: אם התוכנה נכשלה ליצור מודל בחלק כלשהו - סימן שהיה חסר כיסוי או חדות.

תוצר הפוטוגרמטריה: הפלט יהיה מודל תלת־ממדי (פורמט obj/.ply וכד') + קובץ **טקסטורה** (תמונת מפת-מרקם הנמתחת על פני המודל). מודל כזה מאפשר לבצע מדידות - למשל למדוד מרחק מהפיטם, או אחוז שטח הפנים הפגום. אפשר להיעזר בכלים כמו **MeshLab** (בקוד פתוח) כדי לצפות ולערוך את המודל, או אפילו להעלות אותו לכלי צפייה אינטרנטי להמחשה. בחברת 3DScan Solutions מתארים שימוש בתוכנת **Twinspect** לצורך **לכידת פגמים ודיווח** ישירות על מודל פוטוגרמטרי עם טקסטורה ברזולוציה גבוהה¹³. כלומר, לאחר יצירת המודל (למשל ב-RealityCapture), מייבאים אותו לכלי ייעודי כדי לסמן ולתעד פגמים על גבי המודל הווירטואלי. גישה זו מדגישה כיצד המודל יכול לשמש בסיס לסימון **מיקום פגם מדויק** (קו רוחב/אורך על האתרוג) והפקת **דו"ח ויזואלי**.

טיפ מעשי: לאחר סריקת אתרוג וקבלת מודל, כדאי לבדוק את הטקסטורה: אם רואים בטקסטורה את הפגמים בצבע שונה (כתם חום, נקודה שחורה וכו') - סימן שהתהליך הצליח ללכוד אותם. אם הטקסטורה מטושטשת באזור הפגם, זה אומר שכדאי לצלם תמונות תקריב נוספות או לשפר את הפוקוס בפעם הבאה. תמיד אפשר **להוסיף תמונות** חסרות ולהריץ מחדש את האלגוריתם לשיפור המודל.

תיוג (אנוטציה) של מאפיינים הלכתיים באתרוג

כעת, לאחר שיש בידינו תמונות (ואף מודל 3D) של אתרוגים רבים, יש לבצע **תיוג מדויק** של הפגמים והמאפיינים ההלכתיים בכל דגימה. תיוג טוב הוא קריטי: המודל ילמד ממנו מה נחשב פגם, היכן, ומה פוסל אתרוג. יש להגדיר **סכמת תיוג** הכוללת את כל הפרטים הרלוונטיים לפי ההלכה:

- **סיווג כשרות כולל:** לכל אתרוג (או לכל סט תמונות שלו) יש לשייך תווית בינארית - "כשר" או "פסול". תווית זו יכולה להיקבע על בסיס כלל הנתונים של אותו אתרוג. רצוי שההחלטה תתקבל ע"י מומחה הלכתי (רב בעל ידע בדיני ארבעת המינים) שמסמן במערכת התיוג האם האתרוג ראוי לשימוש למצווה. תווית זו תשמש בהמשך לאימון מודל **Classification** אשר מקבל את כלל המידע על האתרוג ומנבא כשרות. עם זאת, **לא די בסיווג הכולל**, כי נרצה גם לזהות מדוע פסול (איזה פגם). לכן נשתמש בנוסף בשאר התגים הבאים.
- **פיטם (סטטוס):** הפיטם הוא ה"עוקץ" בחלקו העליון של האתרוג (ה"שושנתא" בלשון חז"ל). בהלכה, אתרוג שהיה לו פיטם ונפל - **פסול**¹⁴ ¹⁵. לפיכך יש לתייג: האם הפיטם קיים ושלם, שבור, או חסר. זהו שדה שאפשר לתאר כמספר מצבים: "שלם", "נשבר חלקית", "נפל לגמרי". התיוג יתבצע אולי בתמונה ייעודית (למשל צילום של ראש האתרוג) או בסימון על המודל 3D. אפשר גם לבצע זאת כתיוג ברמת האתרוג (attribute): עבור כל אתרוג להזין ערך סטטוס פיטם. במידה והפיטם חסר, מומלץ גם לציין האם זה **חסר טבעי** (ישנם אתרוגים שגדלים ללא פיטם - ואז הם כשרים) או חסר עקב נפילה. מידע זה יילקח בחשבון בקביעת הכשרות.
- **חזזית:** "חזזית" מוגדרת בהלכה כבליטה או גידול על קליפת האתרוג (מעין יבלת או צריבה). הכלל הוא: חזזית גדולה (ששטחה עולה על רוב שטח האתרוג) פוסלת, וכן **אפילו קטנה הממוקמת ב"חותם" (החלק העליון של האתרוג)** פוסלת¹⁶. כמו כן, **שלוש חזזיות נפרדות באתרוג** - **rog** - אפילו קטנות - פוסלות כי נראה "מנומר"¹⁶. יש לתייג הופעת חזזיות: למעשה כל פגם שטח משמעותי (כתם בולט, קליפה גלדית) - לסמן את מיקומו וגודלו. התיוג יכול להיות בפורמט **Bounding Box** סביב האזור הפגוע או אפילו **פוליגון** מסביב לצורתו המדויקת (אם משתמשים בכלי תיוג מתקדם). לכל תיוג כזה יש לשייך קטגוריה "חזזית" (אם זוהה ככזו). מומלץ בנוסף לסמן אם החזזית גבוהה (בולטת) או שטוחה - ייתכן בדרוג, אך אפשר גם להסתפק בכך שהרשת תלמד לפי מראה. מידע המיקום המדויק יאפשר אח"כ לקבוע האם אחת מהן נמצאת באזור ה"חותם" (שליש עליון). ניתן להגדיר בכלל שאת גבול השליש העליון קובעים לפי אורך האתרוג - דבר שניתן למדידה על מודל 3D או אף על תמונה אם כיוולנו בקנה מידה.

- **כתמים ושינויי צבע:** בהלכה, שינוי צבע משמעותי (בעיקר שחור או לבן) דינו כחזית ¹⁷. נקודה שחורה על החותם – פוסלת בכל גודל ¹⁶. לכן יש לתייג נקודות כהות או בהירות חריגות. את רוב הנקודות השחורות הקטנות נהוג לבדוק האם הן לכלוך חיצוני הניתן להסרה, אך במערכת אוטומטית נרצה שהמודל יתפוס אותן כפגמים פוטנציאליים. נסמן אזורים של **שינוי צבע** (שאינו ירוק/צהוב טבעי) ונסווג אותם כ"כתם". אפשר לתת קטגוריות משנה: "כתם שחור", "כתם חום", "כתם לבן" וכו', אם רוצים שהרשת תלמד הבדל (למשל שחור לרוב יותר חמור אם הוא עקב ריקבון ¹⁸). שימו לב שלפי המשנה ברורה, שינוי צבע שפוסל הוא רק כזה שניכר **בהסתכלות רגילה** (לא בזכוכית מגדלת) ¹⁹ – לכן אולי נציב סף: לא לתייג נקודות זעירות מאוד שאינן בולטות. בפועל, המומחה המאנוטט יחליט בכל תמונה האם נקודה מסוימת תסומן.

- **נקבים (חורים/חסרים):** חור באתרוג (חדירה דרך הקליפה) פוסל אפילו קטן ²⁰ ²¹. גם חסרון חלק מהאתרוג (קטיעה) פוסל. לכן כל מקום שבו חלק מהקליפה חסר (נשור, חתוך) – יש לסמן. קטגוריה אפשרית: "חור/חוסר". לרוב זה יתבטא כצורה לא חלקה של הפרי, אולי עם צבע פנימי גלוי. תיוג זה חשוב כי הוא פוסל מיידית, וקל יחסית לזיהוי חזותי.

- **צבע כללי:** אתרוג **ירוק לגמרי** (כצבע עשב) פסול, אלא אם כן התחיל כבר להצהיב מעט ²². ניתן לתייג את הגוון הכללי של האתרוג: למשל כתכונה "צבע" עם ערכים: ירוק-כהה, ירוק-בהיר, צהבהב, צהוב. זאת אפשר לעשות ברמת האתרוג כולו (לדוגמה, לפי אחוזי פיקסלים צהובים בתמונות). המטרה היא שהמערכת תוכל להתריע על אתרוג שעדיין ירוק מדי (אלא אם ידוע שיצהיב עד סוכות – דבר שקשה לחזות, אז נוטים להחמיר). אולי הפתרון הוא פשוט לכלול "ירוק" כמצב פסול במודל הסיווג הסופי.

- **מיתוגים נוספים:** יתכן שצריך לתייג גם **סוגי רקבונות** (אם יש ריקבון או יובש – אתרוג יבש לחלוטין פסול ²³), וכן **האם האתרוג מורכב** (שאלה הלכתית אחרת – קשה לזהות ע"י תמונה, לרוב מתבססים על מסורת/תעודה). אלו כנראה מחוץ לטווח הפרויקט הנוכחי כי דורשים מבחנים אחרים.

לאחר קביעת סכמת התיוג, חשוב לכתוב **הנחיות ברורות לצוות המסמנים** ולוודא אחידות. כל אנוטטור (מתייג) צריך לדעת למשל: איך להבדיל בין חזית לכתם צבע? מה נחשב "בולט"? היכן עובר גבול ה"חותם"? יצירת **מדריך תיוג** עם דוגמאות (תמונה עם סימון נכון) תסייע להשיג תיוג **עקבי ומדויק**. עקביות זו חשובה – היא משפיעה ישירות על ביצועי המודל ²⁴ ²⁵. ניתן גם לקיים **בקרת איכות**: מתייג שני יבדוק חלק מעבודת מתייג ראשון, לוודא שאין פספוסים פגמים או סימונים שגויים ²⁶. במקרה של אי-הסכמה, מערבים את המומחה ההלכתי להכרעה ועדכון ההנחיות. כלי תיוג מתקדמים (כגון CVAT, שיפורט בהמשך) מאפשרים ניהול משתמשים ומשימות, כך שמספר מסמנים יכולים לעבוד במקביל וכל עבודה נבדקת.

כלי תיוג מומלצים: ישנם פתרונות רבים בקוד פתוח לתיוג תמונות ²⁷. עבור הפרויקט שלנו, נרצה כלי התומך גם בתיבות וגם בפוליגונים, ואפשרות להגדיר תוויות קטגוריה מרובות. להלן כמה אפשרויות:

- **CVAT (Computer Vision Annotation Tool)** – כלי פתוח עוצמתי מבוסס-ווב ²⁸. מאפשר לצייר תיבות, פוליגונים, לצבוע פיקסלים (לסגמנטציה), וכן להוסיף תגים אטומיים (כמו "כשר/פסול") ברמת תמונה או אובייקט. CVAT מצוין לצוותים, כי יש בו מנגנון ניהול משימות, חלוקת עבודה, ובסוף אפשר לייצא בפורמטים סטנדרטיים (COCO, YOLO, Pascal VOC) וכדומה.
- **LabelMe** – כלי שולחני/וובי פשוט לתיחום באמצעות פוליגונים ²⁹. מתאים אם רוצים ידנית לסמן צורות לא-מלבניות (למשל קווי מתאר של כתם).
- **LabelImg** – כלי פשוט לתיבות תיחום (BBBox) בפורמט YOLO או PascalVOC ³⁰. יכול להתאים לסימון מהיר של פגמים בתור תיבות, אם כי עלול להיות מאתגר לתחום חזית לא-מלבנית.
- **Label Studio** – פלטפורמה גמישה מאוד המאפשרת גם טקסט, תמונה, וידאו (פתוחה לשימוש עד גבול מסוים) ³¹. ניתן לקנפג בה ממשק ייעודי, למשל תיבת סימון למצבי פטים + כלים לסימון הפגמים.

לא משנה באיזה כלי משתמשים, חשוב **לשמור את התיוגים בגרסה** – כי ייתכן שבהמשך, עם עוד ידע, נשנה קצת את הקטגוריות או נתקן תגים. מומלץ לשמור את כל קובצי האנוטציה (למשל קבצי YOLO txt או COCO JSON) תחת מערכת בקרת גרסאות (אפילו Git פשוט, אם לא גדול מדי, או פתרונות כמו DVC שיזכרו).

דוגמה מוחשית לתהליך תיוג: נניח שיש לנו תמונה ברורה של צד האתרוג, ורואים בה כתם חום בגודל 5 מ"מ באמצע הפרי. המתייג יוצר סביב הכתם תיבת תיחום, בוחר קטגוריה "כתם (חום)". בנוסף, באתרוג זה הפיטם נראה חסר בתמונה – המתייג יפתח תפריט מאפיינים (attributes) לאתרוג הזה ויסמן "סטטוס פיטם: חסר". אחרי שיעבור על כל התמונות של אותו אתרוג ויציין את כל הפגמים הנראים מכל צד, הוא גם יסכם: "כשרות: פסול" (כי פיטם חסר פוסל). כך, ברמת כל אתרוג יש לנו גם אנוטציית פגמים מפורטת וגם תווית סופית.

מבנה מערכת הזיהוי המומלצת

לאחר שיש נתונים מתוייגים, נעצב את **ארכיטקטורת המערכת** שתבצע את הזיהוי בפועל. המערכת צריכה לקבל קלט (תמונות של אתרוג מכל הכיוונים, ו/או מודל 3D שלו) ולהפיק פלט **ברור ומשולב** – חיווי כשרות, רשימת פגמים שנמצאו, ומיקומם. נפרק זאת למספר שלבים מודולריים:

- עיבוד מקדים (Preprocessing):** שלב זה כולל טעינת התמונות ו/או המודל, ושיפורם לפני הזנתם לרשת. אם התמונות צולמו על רקע לא אחיד, פה נבצע הפרדת רקע (למשל בעזרת מסנן צבע – האתרוג צהוב והרקע שחור, או בעזרת מודל segmentor שיוודע להפריד אובייקט רקע). בחדר צילום מבוקר אולי אין צורך, אך ניקוי הרקע יכול להפחית "רעש" למודל. בנוסף, נבצע שינוי גודל (ריסייז) לפי מה שהרשת דורשת – מודלים רבים דורשים קלט ריבועי, למשל 640×640 פיקסלים³², ולכן נגדיל/נקטין עם שוליים מתאימים. נורמליזציה של צבע (למשל סקאלה 0-1 או נירמול לפי ממוצע) – לפי דרישות המודל שנבחר. אם משתמשים במודל 3D, ייתכן צורך **לחץ ממנו נתונים לתכונה** : לדוגמה, לחשב את גובה האתרוג בפיקסלים או במילימטרים, כדי לדעת היכן השליש העליון. אפשר גם לפרוס את הטקסטורה של המודל לתמונה דו-ממדית ולנתח אותה בכלים רגילים (טכניקה: orthophoto או UV texture map).
- זיהוי פגמים בתמונות (Defect Detection):** זהו לב המערכת – כאן נאמן מודל למידת-מכונה שיקבל תמונה (או כמה) של אתרוג ויאתר בה אוטומטית פגמים. יש מספר גישות אפשריות:
- מודל Object Detection דו-ממדי:** למשל YOLOv5/YOLOv8, Faster R-CNN או RetinaNet – שמקבל תמונה ומחזיר רשימת תיבות מסביב לפגמים + קטגוריה לכל פגם. מודלים כאלו ניתן לאמן על התמונות המתויגות שלנו (תיבות/פוליגונים שהכנו). היתרון: YOLO ודומיו יעילים ומהירים, ועם אימון מספיק יוכלו לאתר גם פגמים קטנים. יש לציין שאיתור אובייקטים קטנים הוא אתגר מוכר; ניתן לשפר זאת ע"י צילום ברזולוציה גבוהה ו"חיתוך" התמונה למקטעים (Window Slicing) כדי שהפגם יתפוס יותר פיקסלים³³. ישנן גם טכניקות כמו **SAHI** (Slicing Aided Hyper Inference) שפורסות תמונה גדולה לתת-תמונות לחיפוש אובייקטים זעירים³⁵ ³⁶.
- מודל Segmentation (חלוקה לפיקסלים):** ניתן להשתמש ברשתות כגון **Mask R-CNN** או **U-Net** כדי לבצע סגמנטציה לסוגי שטח על האתרוג: פיקסלים של רקע, פיקסלים של אתרוג תקין, ופיקסלים של פגם. כך נקבל "מסכת" אזורים פגומים. זה יכול לתת מיקום וצורה יותר מדויקת מתיבה. עם זאת, אימון סגמנטציה מצריך אנוטציית מסכות מדויקת – מלאכה כבדה יותר מתיבות. אפשר לשלב: למשל YOLOv8 מכיל גם ראש של **Instance Segmentation** שיכול לתת מסכה לכל אובייקט מזהה.
- ניתוח על גבי המודל 3D:** גישה אחרת, פחות שגרתית אבל מעניינת, היא לנסות לנתח את **מפת הטקסטורה** של האתרוג. לאחר שיש לנו את הטקסטורה (תמונה דו-ממדית המייצגת את פרישת פני האתרוג), אפשר להריץ עליה אלגוריתם לגילוי כתמים (למשל ע"י הבדלי צבע מהצבע הבסיסי). תוכנות מסחריות לפוטוגרמטריה לפעמים מספקות **Ortho Projection** – תמונה אורכוגרפית של פני האובייקט. בעזרת תמונה זו ניתן לגלות הבדלים ולמפותם חזרה למודל. אמנם, פתרון זה דורש כתיבה יעודית ואינו "מהמדף" כמו מודלי DL מוכנים. בפועל, כנראה שיופעל מודל גילוי על התמונות הרגילות, והמודל התלת-ממדי ישמש רק לעיבוד מיקום כפי שיפורט בהמשך.

במערכת מיטבית, אפשר אף **לשלב מודלים** : למשל מודל ראשוני שמאתר את הפרי ומסיר רקע, מודל שני שמאתר פגמים, ואולי מודל שלישי שמבצע סיווג סופי. שילוב שלבי עיבוד מקובל בפרויקטים כאלה כדי לחלק את הבעיה לתת-בעיות מטופלות היטב (מודולריות).

1. **חישוב מיקום פגם גלובלי:** לאחר שקיבלנו זיהוי פגם בתמונה, צריך להבין **היכן על האתרוג** הפגם נמצא. מכיוון שיש לנו מודל תלת-ממדי, אפשר למפות כל פגם למרחב 3D: אם אנו יודעים באיזו תמונה התגלה פגם, ונשתמש בנתוני הכיול של הפוטוגרמטריה (נניח התוכנה מספקת לנו את מטריצות המצלמה לכל תמונה), נוכל להקרין את מיקום התיבה/מסכה לאורך קרני הראייה במודל ולמצוא את הקואורדינטות על המשטח שהתאימו לאותו פיקסל. זהו תהליך הנקרא **Triangulation** – התאמת נקודת תמונה לשטח אובייקט. התוצאה: לכל פגם יהיו קואורדינטות (x,y,z) על פני המודל. כעת קל לחשב למשל את **המרחק מהפיטם** (שהוא קצה המודל) – אם המרחק קטן משליש האורך הכולל, נסמן שהפגם ב"שליש העליון (חותם)". בצורה זו המערכת תוכל להחליט: "יש חזזית באזור החותם" – מה שלפי הכללים הופך מיד את האתרוג לפסול ¹⁶. לחילופין, אם אין מודל 3D או לא רוצים חישוב כזה, אפשר פשוט להתבסס על התמונות: אם יש תמונה שצולמה ככה שרואים בה גם את הפיטם וגם את הפגם בקו ראייה, אז כנראה זה קרוב לפיטם. אבל זה הרבה פחות מדויק. לכן, **המודל התלת-ממדי הוא כלי מועיל מאוד להערכת מיקום יחסי**.

2. **סיווג כשרות (Classification):** בשלב האחרון, על בסיס כל המידע שנאסף (פגמים שסומנו, קטגוריותיהם, מיקומם), צריך להחליט האם האתרוג כשר. ניתן לגשת לכך בשני אופנים:

3. **חוקים לוגיים (Rule-based):** ליישם ישירות את כללי ההלכה על הפגמים שמצאנו. למשל: אם `pitom_status == "חסר"` אז כשרות="פסול"; אחרת, אם יש `count(חזזית) >= 3` אז פסול; אם `sumArea(פגם)/surface_area > 0.5` אז פסול (רוב שטח הפרי פגום); אם צבע = ירוק כהה אז פסול; וכן הלאה. היתרון של גישה זו – שקיפות ותואמות מלאה לכללי ההלכה הידועים ¹⁶ ¹⁷. למעשה, המערכת משמשת כ"מסייע" אוטומטי שמיישם את פסיקות הפוסקים.

4. **מודל למידת מכונה לסיווג:** אפשר גם לאמן רשת (למשל ResNet או EfficientNet) שמקבלת את תמונות האתרוג (או מידע מעובד כמו מפת פגמים) ופולטת ישירות "כשר" או "פסול". זה דורש הרבה דוגמאות מתוגות (כשר/פסול) כדי שהרשת תלמד בעצמה את הכללים. ייתכן שהרשת תלמד לבד את החוקיות (למשל תתן משקל גדול לאזור הפיטם), אבל חסרונה הוא **חוסר שקיפות** – לא נדע בדיוק איזה פגם גרם להחלטה. שילוב גישות עדיף: להשתמש במודל הסיווג הסטנדרטי בנוסף לחוקים. אם שניהם מסכימים – מצוין. אם יש סתירה, אפשר למשל להציף להחלטת אדם. בשלב הראשוני, קרוב לוודאי שנגדיר את הכשרות על סמך החוקים (כי כבר הפקנו מהזיהוי את כל הנתונים הדרושים להסקה).

5. **יצירת הפלט וההצגה:** המערכת תפיק דו"ח סופי. הפלט המינימלי: "כשר" או "פסול". אך כדי להיות שימושי, רצוי פלט מפורט יותר: **רשימת נימוקים ופגמים**. למשל, עבור אתרוג שזוהה כפסול, הדו"ח יכול לפרט: "פסול: הפיטם חסר, וכן נמצאה חזזית באזור החותם." או "כשר אך עם הערה: כתם חום קטן בחלק התחתון (אינו פוסל)". אפשר להציג **תמונות עם סימונים** – למשל לצייר תיבה/מסגרת סביב הפגם שנמצא, כדי שמשתמש אנושי יוכל לאשר את תקפות הזיהוי. במערכת מתקדמת, ניתן אף לספק **מודל תלת-ממדי אינטראקטיבי** עם סימונים: המשתמש יכול לסובב מודל אתרוג ורואה עליו נקודות/אזורים מסומנים באדום היכן שיש פגם. זה מקל על בדיקה ידנית במקרה הצורך. לחלופין, מצגת 3-4 תמונות עיקריות של האתרוג מכל צד עם סימון הפגמים על כל תמונה תספיק.

כמו כן, תיתכן אינטגרציה **למסד נתונים** לשמירת תוצאות: עבור כל אתרוג (מזוהה לפי ברקוד/ID) תישמר תוצאת הבדיקה והממצאים. כך ניתן לעקוב אחר איכות האתרוגים למשל אצל ספק, ולהפיק סטטיסטיקות. המערכת יכולה לפעול באופנים שונים: - **אופליין (אצווה):** צילום במספר תחנות, ניתוח ע"י מחשב חזק מרכזי, והפקת דוחות. - **Online Real-time:** תא צילום ובו מצלמות מכל הכיוונים, אתרוג מוכנס, ובתוך שניות מתקבל פלט (דורש חומרה חזקה או מודלים יעילים כמו YOLO בגרסאות קלות). - בכל מקרה, הארכיטקטורה תישאר דומה – אולי עם שינויים בדגש על מהירות לעומת דיוק.

טיפ: כבר במהלך הפיתוח מומלץ **להדגים אב-טיפוס** על אתגרים בודדים – למשל להזין תמונות של אתרוג עם פיטם שבור ולוודא שה-Pipeline מזהה את הפגם הנכון ומכריז "פסול". תיקוף ידני של מקרים מבחן יעזור לבטח את החוקים והמודלים לפני שמריצים על אוסף גדול.

טכנולוגיות וכלים מומלצים לאורך השרשרת

כדי לממש את המערכת, נשלב מספר כלי תוכנה וחומרה. נפרט לפי שלבי השרשרת:

שלב הצילום והסריקה: - ציוד צילום: מצלמה דיגיטלית עם חיישן איכותי (למשל DSLR Canon/Nikon/Sony) עם עדשת מאקרו) תיתן תוצאות מיטביות. לחלופין, סמארטפון מודרני עם מצלמה 12MP+ ומצב Manual עשוי להספיק. שימוש ב-**חצובה** ותאורה קבועה חיוני כדי למנוע סטיות. אם התקציב מאפשר – **עדשת מאקרו** יכולה לתפוס פגמים זעירים היטב. כמו כן, **פלטת מסתובבת ממונעת** (Turntable) תיעיל את הצילום מזוויות קבועות. יש כאלו עם סימוני זווית או שלט למחזור אוטומטי של עצירות (כל 5° למשל). - תאורה ורקע: כדאי לרכוש **Lightbox** (תא צילום) בעל קירות LED לבנים. זה נותן אור מפוזר ורקע מתחלף (בדרך כלל מסופק רקע שחור ולבן קטיפתי). הקטיפה בולעת אור וכך הרקע יוצא אחיד ללא החזרות ¹¹. רקע שחור כנגד אתרוג צהוב-בהיר מספק קונטרסט מצוין. ודאו שהרקע גדול מספיק כדי שלא ייכנסו בשולי הפריים דברים מעבר (ניתן לזום או לקרב את המצלמה כך שהאתרוג ממלא את רוב הפריים ³⁷).

שלב בניית ה-3D: - תוכנות פוטוגרמטריה: כאמור, בקוד פתוח בולט **Meshroom** (דורשת GPU CUDA, מאוד נוחה לשימוש, והתוצאות טובות לפריטים קטנים). **COLMAP** היא ספרייה ו-CLI עם אלגוריתמים מעולים – דורשת יותר ידע טכני לשימוש. אם מוכנים להשקיע בכלי מסחרי: **Agisoft Metashape** מאוד פופולרית וידידותית, או **RealityCapture** שהיא סופר מהירה (אבל רצה רק על Windows, ותשלום חודשי או לפי נקודות). אפשר לשקול גם **Photogrammetry apps** בטלפון (כגון Polycam, Trnio) – אך לא סביר שיתנו רזולוציה מספיקה לפגמים קטנים. עבור הדמיה מהירה אולי, אך למערכת מקצועית עדיף PC. - כלי ניתוח 3D: לאחר יצירת המודל, **Meshlab** (חינמית) מאפשרת לבצע מדידות, לראות קואורדינטות ולתקן מעט רשתות. גם **CloudCompare** (חינמי) שימושי למדידת מרחקי נקודות (ניתן למדוד מרחק מהפיטם לפגם בקלות בענן נקודות). לציון סימונים על המודל התלת-ממדי, אפשר להשתמש בכלי עריכת Mesh, אך פתרון אלגנטי: לטעון את המודל לתוכנה גרפית כמו **Blender** ולמקם "סיכות" או סימונים. עבור תצוגה ללקוח, אפשר להעלות ל-Sketchfab (פלטפורמת שיתוף 3D) ולסמן Annotations אינטראקטיביים.

שלב התייג: - כפי שפורט, **CVAT** הוא פתרון טוב לצוות. ניתן להקים שרת CVAT פנימי, להעלות אליו את כל התמונות, להגדיר פרויקט עם תוויות ("Pitom", "Hazazit", "Ketem", וכו' + attributes כמו Koshers: yes/no), ולחלק משימות בין מתייגים. - אם הצוות קטן ויעבוד מקומית, **LabelImg** עשוי להספיק (הוא ייצור קבצי xml או txt לכל תמונה). במידה ורוצים סגמנטציה, **LabelMe** או **VGG Image Annotator (VIA)** יכולים לתת מסכות. - **Labelbox / Supervisely / Makesense.ai** – פלטפורמות נוספות (חלקן Freemium) עם ממשקים ידידותיים, יכולות AI-assisted (למשל מודל מקדים שמנחש תיבה), ושיתוף פעולה. **Labelbox** מסחרי אך חינמי לכמות קטנה של נתונים – יכול להתאים לפיילוט. **Supervisely** מצוין לסגמנטציה ויש לו אפילו פלאגין לצפות ב-3D point cloud, אך הגרסה החינמית מוגבלת.

שלב אימון מודלי ML: - פלטפורמת פיתוח: מרבית אנשי ה-Data Science יבחרו בסביבות Python עם **PyTorch** או **TensorFlow**. PyTorch פופולרי למחקר וקל לבצע בו Transfer Learning. למשל, אפשר לקחת מודל YOLOv5 מותאם-מראש (pre-trained) על COCO, ולהמשיך לאמן אותו על תמונות האתרוגים שלנו (fine-tune) – כך הוא ילמד לזהות את הקטגוריות החדשות ("חזית", "כתם" וכו') ⁵. **YOLO v8** (Ultralytics) הוא ספרייה נוחה במיוחד – מספקת ממש API פשוט להגדיר נתיב לנתונים ולאמן. - מודלים מתאימים: עבור **זיהוי (Detection)** – משפחת YOLO (v5, v7, v8), מציעה איזון טוב בין דיוק למהירות. קיימים דגמים במגוון גדלים (nano, small, medium, large) – אפשר להתחיל מדגם גדול כדי לקבל דיוק מרבי, ואח"כ לבדוק אם דגם קטן יותר נותן די והותר. לחלופין, **Detectron2** של פייסבוק יכול לאמן Mask R-CNN עבור זיהוי + סגמנטציה – זה ייתן אזורי פגם מדויקים. עבור **סיווג (Classification)** – אפשר להשתמש ברשת כמו ResNet50 או EfficientNet-B3, לאימון על תמונות שלמות של אתרוג (או על שילוב תכונות). אם נרצה להזין כמה תמונות של אותו אתרוג כמקשה לסיווג, אחד הפתרונות הוא יצירת **Embedding משותף**: למשל, להריץ כל תמונה דרך רשת חזקה (ResNet) ולקבל וקטור תכונות, ואז למזג את הוקטורים מכל זוויות האתרוג (ע"י ממוצע/מקסימום) ולהעביר למודל החלטה. זה מתקדם יותר, אך אפשרי. - טיפול באובייקטים קטנים: בהנחה שפגמים כמו נקודה שחורה יכולים להיות זעירים (כמה פיקסלים בלבד), חשוב להתאים את האימון: להשתמש ברזולוציית קלט גבוהה (אפשר 1024x1024 במקום 640x640, אם המודל תומך) – במחיר זמני חישוב ארוכים יותר. אפשר גם **להגדיל**

באופן מלאכותי את האזורים עם פגמים בתמונות (Crop & Resize) כדי ללמד את הרשת טוב יותר מה לחפש. ספריות כמו **SAHI** שהזכרנו מסוגלות בזמן **הסקה** (Inference) לחתוך תמונה גדולה לתת-ריבועים למשל 300×300 ולרוץ על כל חתיכה, מה שמשפר ריקול של פגמים קטנים ³³ ³⁶. - אימות ובדיקה: חשוב לשמור חלק מהנתונים כסט אימות/בדיקה (לא להשתמש בהם באימון) על מנת לוודא שהמודלים עובדים טוב על אתרוגים שלא "ראו". נעקוב אחרי metrics כמו Precision, Recall לכל קטגוריה (למשל, recall של גילוי חזיות בחותם צריך להיות מאוד גבוה - אסור לפספס). במקרה של ביצועים לא משביעי רצון בקטגוריה מסוימת, נחזור לנתונים - ייתכן שצריך לתייג עוד דוגמאות מסוג זה, או לבצע Augmentation מתאים. - סביבת חומרה: אימון רשתות עמוקות דורש GPU חזק (NVIDIA עם 12GB+ VRAM רצוי). אם לא בנמצא מקומית, אפשר לשקול שירותי ענן (AWS, GCP) או פלטפורמות כמו Google Colab Pro. מודל YOLOv5large למשל יכול להתאמן בכמה שעות על ~2000 תמונות עם GPU מתקדם.

שלב הרצה (Deploy) ויצירת דוח: - תשתית תוכנה: לאחר אימון, צריך לשלב את הכל לכדי תוכנה אחת. סביר שנכתוב קוד Python שמשתמש בספריות כמו **OpenCV** (לעיבוד תמונה בסיסי), **PyTorch** (לטעינת המודלים המאומנים ולהרצתם על תמונות חדשות), וייתכן **Open3D** או ספריית תלת-ממד עבור שלב המיפוי המרחבי. - אינטגרציה עם ממשק משתמש: אם התוצאה אמורה להיות אפליקציה, ניתן ליצור ממשק Web (למשל Flask או FastAPI יריצו את המודל ויחזירו תוצאות לתצוגה ב-Webpage). לחלופין ממשק Desktop ב-PyQt לטכנאי מעבדה. - ד"ח ותצוגה: אפשר להפיק **קובץ PDF** אוטומטי עם תמונות המסומנות וטקסט מסכם. בסביבה תעשייתית, אפשר גם לשמור ל-DB ולספק **Dashboard** סטטיסטי. - תלת-ממד: עבור הצגה אינטראקטיבית, אפשר לשלב **Viewer** תלת-ממדי (יש WebGL ושאר טכנולוגיות - למשל להטמיע דגם ב-HTML עם ספריית *three.js*). אם זו יותר מדי עבודה, לפחות לצרף לתוצר **קובץ obj**. **עם טקסטורה**, כדי שאם מישהו ירצה - יוכל לפתוח ולראות (אפשר אפילו לצרף קובץ שכולל סימון כוירטואלי - אך זה מורכב).

סיכום ביניים של רכיבי המערכת (כלים):

שלב	פתרונות קוד פתוח (מומלץ)	פתרונות מסחריים/מוכנים
צילום	Lightbox, מצלמת DSLR/סמארטפון, חצובה, Turntable	ציוד מקצועי לצילום מוצר (אמזון וכו')
פוטוגרמטריה	Meshroom, COLMAP, OpenMVG+MVS	Agisoft Metashape, RealityCapture
תיגון נתונים	CVAT, LabelImg, LabelMe, Label Studio	Labelbox, Supervisely
אימון מודלים	PyTorch, Ultralytics YOLO, Detectron2, TensorFlow	שירותי AutoML בענן (Google AutoML Vision וכד')
ניתוח 3D	Meshlab, CloudCompare, Open3D (Python)	Twinspect, Geomagic (מדידה ידנית)
ניהול גרסאות	³⁸ Git + DVC (Data Version Control)	פתרונות SaaS (Weights&Biases) לדוגמה לניסוי

כדאי לשים לב שבפרויקט כזה נשתמש ברוב בכלי הקוד הפתוח, המאפשרים גמישות והתאמה לצרכים ההלכתיים הייחודיים שלנו. שילוב כלים דורש ידע טכני מגוון בצוות - ראו בסעיף הבא.

דגשים ארגוניים וניהול פרויקט הנתונים

הצלחת המיזם תלויה לא רק בטכנולוגיה אלא גם בהתנהלות הצוות ובניהול הנתונים והגרסאות לאורך הדרך. הנה מספר הנחיות ארגוניות:

צוות מולטי-דיסציפלינרי: רצוי להרכיב צוות **קרוס-פונקציונלי** שבו לכל תחום יש נציג: - מומחה הלכתי (לייעוץ בהגדרת הקריטריונים, בדיקת תיוגים שניים במחלוקת ולהכרעה מה נחשב פסול בגבולות האפורים). - מומחה צילום/הנדסה (להקמת מערך הצילום והפוטוגרמטריה, כתיבת פרוטוקול צילום כך שאנשי השטח יוכלו לצלם בצורה סטנדרטית). -

מדעני נתונים/מהנדסי ראייה ממוחשבת (לפתח את המודלים, לנתח ביצועים, לכוון איסוף נתונים נוסף במקומות חלשים). - מתייגים/אנוטטורים (יכולים להיות סטודנטים לשיבה המכירים הלכה בסיסית, לאחר הכשרה קצרה על התוכנה, או עובדי QA מיומנים). - מנהל פרויקט/מהנדס מערכת (לתאם בין כולם, לקבוע לוחות זמנים ולוודא שילוב חלק של הרכיבים).

גרסאות נתונים ומתודולוגיית פיתוח: מומלץ לאמץ שיטות מעולם ה-ML ops. כל **מקבץ נתונים** משמעותי שנאסף - לתעד כ"גרסה" (למשל "Dataset v1 - 500 אתרוגים ראשונים מאילת", "Dataset v2 - הוספת 200 אתרוגים ירוקים"). יש להשתמש בכלי **בקרת גרסאות נתונים** כמו DVC כדי לשמור snapshot של הנתונים והתיגים בכל שלב ³⁸. DVC למשל מאפשר לשמור מטא-נתונים של הקבצים הגדולים ב-Git, ולהפריד את הקבצים עצמם באחסון חיצוני, כך שניתן לשחזר בדיעבד כל סט נתונים שהודרך עליו מודל מסוים. כך אם פתאום המודל בגרסה 5 מדורדר - אפשר להשוות לנתונים של גרסה 4 ולגלות אולי שתיקוני תיוג מסוימים השפיעו.

כמו כן, יש לתעד גם **גרסאות מודלים** - כל פעם שמאמנים רשת ושומרים משקלות (weights), לתייג אותן (למשל "model_v1.0_trained_on_dataset_v1"). מערכת ניהול ניסויים (Experiments) כמו MLflow או אפילו רישום פשוט בטבלת גוגל שיטס - תעזור לעקוב איזה מודל השיג איזה דיוק ובאילו נתונים השתמש.

תיאום ושיתוף ידע: רצוי לקיים מפגשים סדירים בין כלל הצוות: המתכנתים והאנוטטורים צריכים לשמוע מהמומחה ההלכתי על מקרים גבוליים (למשל: "נקודה חומה בגודל ראש סיכה בחותם - לפסול או לא?") - החלטה כזו צריכה להילקח בצורה עקבית ע"י המערכת, ולכן על כולם להסכים איך לתייג ולהחליט). כך מפתחים **אחידות בהבנה** שמחלחלת לנתונים ולמודל. בנוסף, אם מדען הנתונים מזהה שהמודל מתבלבל בין "חזזית" ל"כתם לכלוך", אולי צריך לשפר את הגדרת הקטגוריות או להוסיף קטגוריה נוספת ("לכלוך שאינו קבוע"). שיחות כאלה בין תחום התוכן לתחום הטכני הן קריטיות.

תיעוד התהליך: לכל שלב יש לתעד את המתודולוגיה וההגדרות: מסמך הנחיות הצילום (מצלמה, מרחק, תאורה - כדי שאם עוד צלם יצטרף הוא יעשה אותו דבר), מסמך הגדרות תיוג (עם תמונות דוגמה לתיוג נכון vs שגוי), וכן יומן ניסויים למודלים (איזו רשת ניסו, אילו היפרפרמטרים). תיעוד זה חשוב לא רק לצורך שיתוף פנימי, אלא גם אם נדרשת **הוכחה מסביבת כשרות** - להראות שהתהליך אמין ועקבי.

ניהול הנתונים בפועל: אם עובדים עם מערכת תיוג כמו CVAT, יש יתרון - היא עצמה שומרת גרסאות של האנתוטציות וניתן לייצא בכל עת. כדאי להגדיר **Naming convention** ברור למקרי מבחן: למשל מספר סידורי לאתרוג + ציון השנה/עונה. את התמונות והמודלים הגולמיים חשוב לגבות. נפחי הנתונים יכולים להיות גדולים (200 תמונות × למשל 1000 אתרוגים = 200k תמונות, אם כל תמונה ~5MB זה כבר 1TB). צריך פתרון אחסון נגיש בצוות - NAS פנימי או אחסון בענן עם גישה נוחה.

שמירת קשר עם התפתחויות: הפרויקט שואב ידע מתחומי ה-Deep Learning, ראייה ממוחשבת והלכה. ראוי שצוות ה-Data Science יתעדכן במחקרים דומים (למשל, עבודות על **זיהוי פגמים בפירות** - יש לא מעט מחקרים חקלאיים על זיהוי פגמים בתפוחים, הדרים וכדומה, שיכולים לתת רעיונות באוגמנטציה או ברשתות מותאמות למרקם ³⁹ ⁴⁰). גם תחום הפוטוגרמטריה מתקדם - ייתכן שבהמשך נשלב טכניקות כמו **Neural Radiance Fields (NeRF)** ליצירת מודל יעיל יותר, אם יהיה כדאי. עדכונים הלכתיים (אם יש דעות חדשות/נדירות לגבי מה לפסול) גם צריכים להגיע לצוות.

בדיקות וליטוש סופי: לפני הטמעה, יש לתכנן **פיילוט**: להריץ את המערכת על, נאמר, 100 אתרוגים ולבדוק ידנית את התוצאות מול פוסק הלכה. כל טעות סיווג - לנתח את הסיבה: פספס פגם? זיהה פגם שלא קיים (false positive)? תיקן לא נכון (חשב שכתם קטן זה חזזית חמורה?). בהתאם - לשפר את המודל או את הנתונים. ייתכן וכיול פרמטרים כמו confidence threshold של המודל (הסף מעליו סימון נחשב לזיהוי) יידרש.

לסיום, שילוב טכנולוגיות הראייה הממוחשבת בהלכה הוא אתגר מרתק. המערכת שתוארה תסייע **להאיץ** ולשפר את תהליך מיון האתרוגים, לספק אובייקטיביות עקבית, ולתת למשגיחי כשרות כלי עזר חדשני. בעזרת ארגון נתונים קפדני, אימון מודלים מתאים וטכנולוגיות 3D, אפשר להגיע **לזיהוי מדויק מאוד** של פגמי אתרוג - עד רמת נקודה זעירה בראש האתרוג - ולפסוק את כשרותו בלחיצת כפתור. המפתח הוא שיתוף פעולה בין מומחי התחום למדעני הנתונים, וניהול נכון של כל חלקי הפרויקט. ¹⁶ ³⁸

-
- .Small object photogrammetry – how to take photos – Prof. Peter L Falkingham 37 12 11 8 4 3 2 1
/https://peterfalkingham.com/2019/01/16/small-object-photogrammetry-how-to-take-photos
- How to Organize a Dataset for Computer Vision? | by Alvaro S. Marcus | Medium 32 7 6 5
https://medium.com/@alvaro.marcus/how-to-organize-a-dataset-for-computer-vision-e28afb89723f
- Defect detection using photogrammetry - 3DScan Solutions GmbH 13 10 9
/https://www.3dscan-solutions.de/en/digitization/photogrammetry/Defect-detection-using-photogrammetry
- Etrog - Halachipedia 23 22 21 20 19 18 17 16 15 14
https://halachipedia.com/index.php?title=Etrog
- Data Collection and Annotation Strategies for Computer Vision 31 30 29 28 27 26 25 24
/https://docs.ultralytics.com/guides/data-collection-and-annotation
- How to Detect Small Objects - Voxel51 36 35 34 33
https://voxel51.com/blog/how-to-detect-small-objects
- Versioning Data and Models | Data Version Control · DVC 38
https://dvc.org/doc/use-cases/versioning-data-and-models
- ... An image segmentation approach for fruit defect detection using k 39
https://link.springer.com/article/10.1007/s40595-014-0028-3
- ... An accurate detection and segmentation model of obscured green 40
https://www.sciencedirect.com/science/article/abs/pii/S0168169922003015